

Сверточные нейронные сети (часть 2)

Занятие 15

Computer Vision

Лектор
Ян Колода

r_d

DATA SPLIT: TRAIN — VALIDATION — TEST

- **Train**
 - Данные, которые используются для тренировки сетей
- **Validation**
 - Данные, которые используются для оптимизации гиперпараметров
 - Не используются в процессе тренировки сетей (модель их «не видит»)
- **Test**
 - Данные, которые используются для окончательной оценки эффективности
 - Не используются в процессе тренировки сетей (модель их «не видит»)

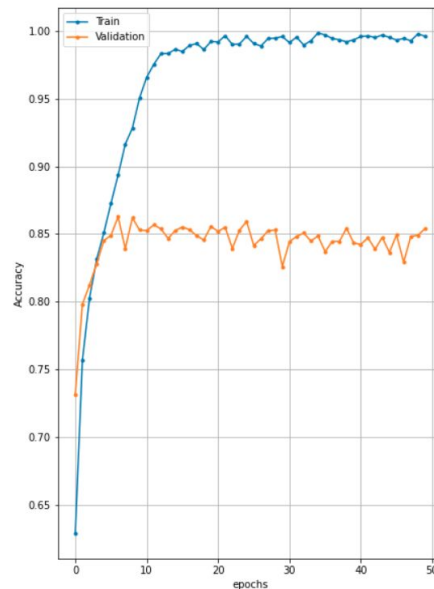
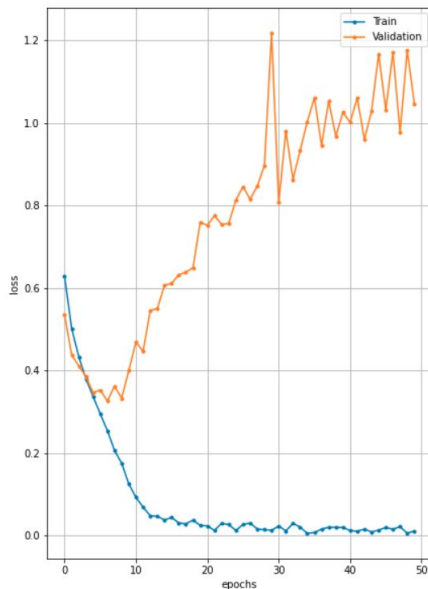
ПРОБЛЕМА ПЕРЕОБУЧЕНИЯ (OVERFITTING)

Learning vs memorizing

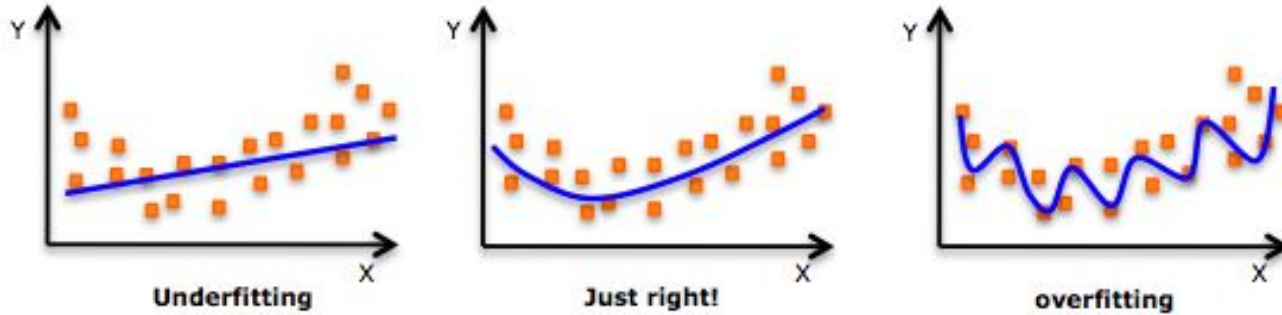
- Low generalization



<https://twitter.com/vardi/status/997851142091628544>



OVERFITTING



An example of overfitting, underfitting and a model that's "just right!"

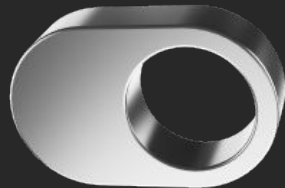
<https://medium.com/analytics-vidhya/the-perfect-fit-for-a-dnn-596954c9ea39>

РЕГУЛЯРИЗАЦИЯ

Как бороться с переобучением?

- С помощью **регуляризации**

Метод добавления некоторых дополнительных ограничений к условию с целью решить некорректно поставленную задачу или предотвратить переобучение. Эта информация часто имеет вид штрафа за сложность модели.

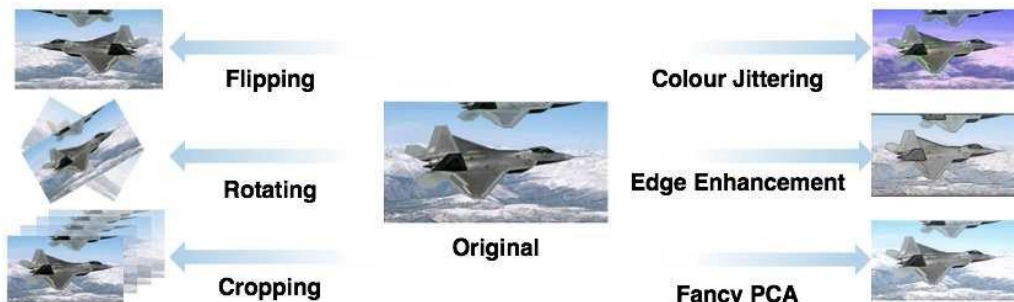
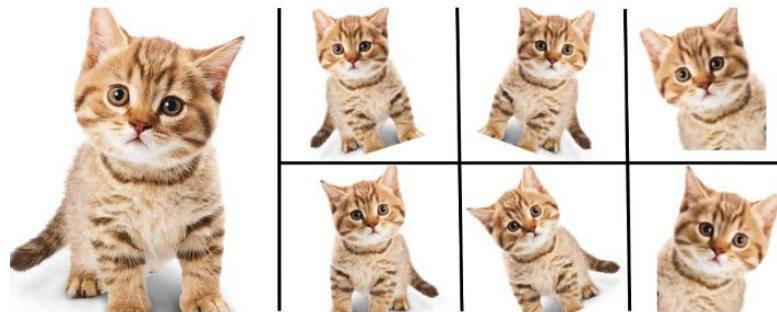


DATA AUGMENTATION

Модификация данных для тренировки:

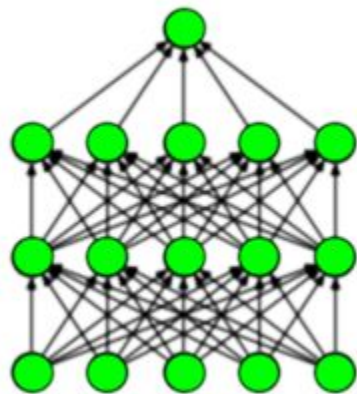
- Blurring
- Gamma correction
- Histogram equalization
- Flipping
- Rotation
- Cropping
- Affine transform
- Noise
- ...

([link 1](#)) ([link 2](#))

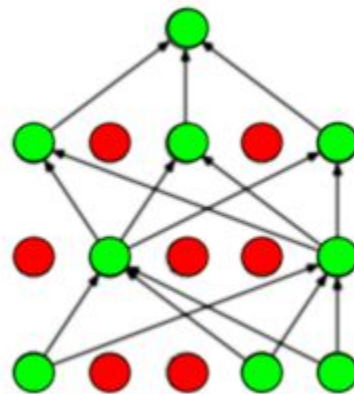


DROPOUT

Нейроны **случайно** исключаются во время обучения

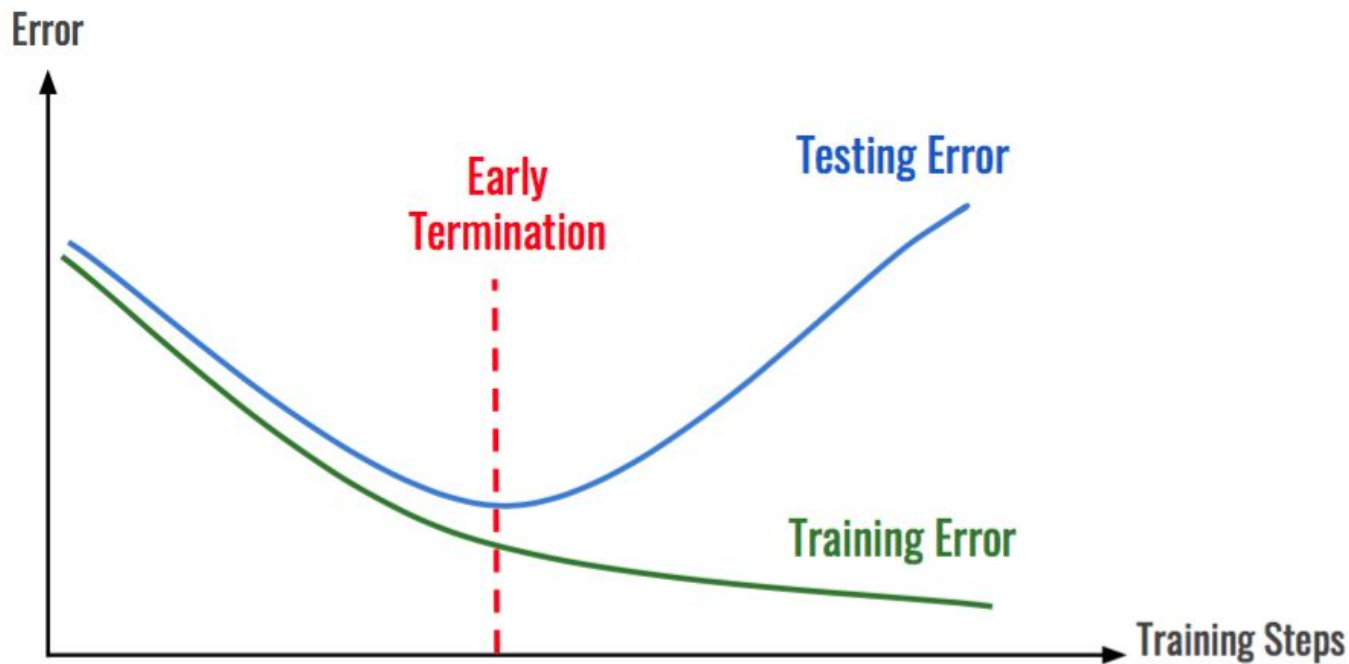


(a) Standard Neural Net



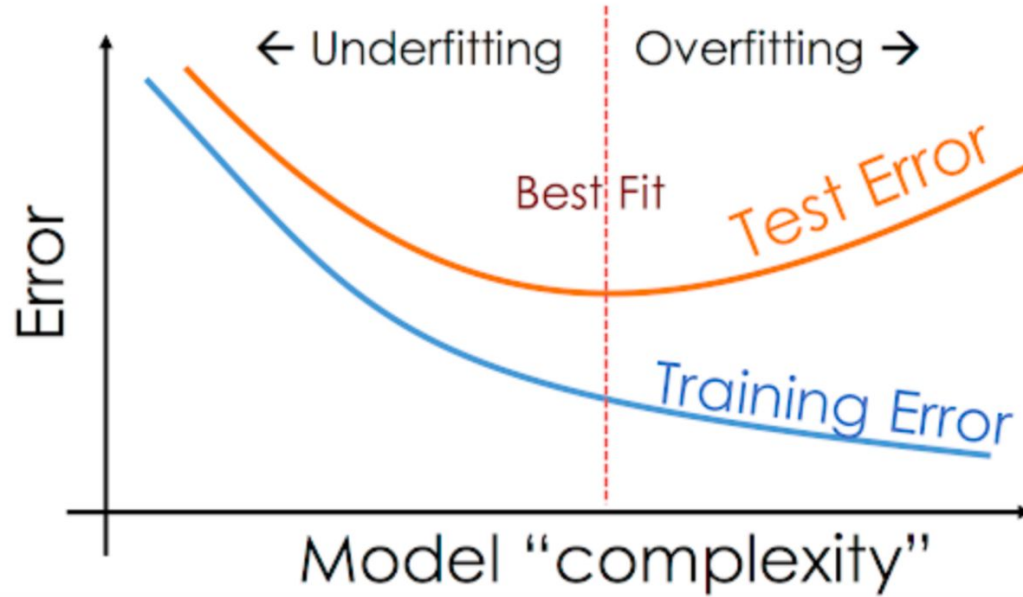
(b) After applying dropout.

EARLY STOPPING



(source)

MODEL COMPLEXITY



(source)

BATCH NORMALIZATION

Normalization (**adaptive** re-centering and re-scaling) of intermediate layers

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

([paper](#))

WEIGHT REGULARIZATION

Также известен как weight decay

`tf.keras.regularizers.Regularizer`



TensorFlow 1 version



View source on GitHub

Regularizer base class.



View aliases

Regularizers allow you to apply penalties on layer parameters or layer activity during optimization. These penalties are summed into the loss function that the network optimizes.

Regularization penalties are applied on a per-layer basis. The exact API will depend on the layer, but many layers (e.g. `Dense`, `Conv1D`, `Conv2D` and `Conv3D`) have a unified API.

Q&A

???



ЗАВЖДИ Є КУДИ
ЗРОСТАТИ